Learning-Based Anomaly-Tolerant Resource Management in Cloud-Edge Continuum

Master Thesis

submitted in partial fulfillment for the degree of master of science

Thomas Erhard
15081931

Master Information Studies
data science
Faculty of Science
University of Amsterdam

Submitted on 30.06.2024

	UvA Supervisor
Title, Name	Hongyun Liu
Affiliation	UvA Supervisor
Email	h.liu@uva.nl

ABSTRACT

As cloud computing increasingly becomes critical to business operations, the need for efficient resource management in cloud environments has intensified. While heuristic methods have traditionally dominated this area, the use of reinforcement learning (RL) strategies has emerged as a promising alternative. This research explores the use of deep reinforcement learning (DRL) to not only improve resource distribution within the cloud-edge continuum but also to implement anomaly detection for increased efficiency. Our model, DeepRM_2, builds upon state-of-the-art frameworks. It leverages a convolutional neural network (CNN) combined with imitation learning to more precisely capture the state space. The goal is to find an optimal policy that maximizes resource utilization while minimizing job slowdown. Compared to its predecessor, DeepRM_ECO, DeepRM_2 demonstrates faster convergence and a remarkable 50.29% reduction in mean job slowdown. Additionally, it excels in detecting anomalies, achieving an F1 score of 0.54, 0.65, and 0.76 for anomaly rates of 0.10, 0.25, and 0.50, respectively, compared to 0.22, 0.44, and 0.61 for DeepRM_ECO, further enhancing overall system efficiency.

KEYWORDS

Deep reinforcement learning, job scheduling, anomaly detection, convolutional neural network, cloud-edge continuum

GITHUB REPOSITORY

https://github.com/Jinobey/DeepRM_2

1 INTRODUCTION

In recent years, cloud computing has become increasingly prevalent, offering on-demand access to scalable computational resources with minimal initial investment and flexible, short-term options. This model eliminates the need for individual data centres, allowing users to tap into computing services as needed [3, 4, 7]. However, its integration into the broader computing ecosystem has not been without challenges.

One significant issue is the latency encountered at the network's edge, which has propelled the rise of edge computing as a strategic complement [12]. By processing latency-sensitive tasks, such as real-time video analytics, at edge nodes closer to users, edge computing helps mitigate latency issues. However, these nodes operate at lower computation speeds. Additionally, edge computing can reduce bandwidth usage by processing data locally, minimising the need for large data transfers to central cloud servers and thus improving overall system efficiency [10, 23, 31].

Achieving the optimal distribution of computational resources between cloud and edge to meet end-user demands is crucial, illustrating a typical case in resource management. Resource management challenges are prevalent across various domains, such as production planning, computer management, or logistics [21], and are traditionally addressed through heuristic methods, which aim to find a near-optimal solution within a feasible timeframe. Traditional scheduling heuristics, such as First Come First Serve (FCFS) [6], Shortest Job First (SJF) [11], or Weighted Round Robin (WRR) [22], have been foundational in this effort. However, the emergence of machine learning, especially reinforcement learning (RL), presents a new opportunity to address such tasks [40, 42].

DeepRM [27] emerged as a pioneering approach that demonstrated the potential of deep reinforcement learning (DRL) for effective resource management in cloud computing environments. Building upon the foundational work of DeepRM, the subsequent development of DeepRM_plus [17] further advanced this field by integrating a convolutional neural network (CNN) and imitation learning. This enhancement improves the model's training efficiency and decision-making capabilities for faster convergence. Additionally, research efforts have expanded to address resource management at the network edge, employing a model-free DRL approach [32, 41].

Another critical challenge within the increasingly complex domain of cloud computing is the need to identify anomalies and develop inherently fault-tolerant systems. While there has been considerable research in the field of anomaly detection [1], most of them focus on identifying malware and Denial of Service (DoS) attacks [13, 24, 29, 39]. The first effort to implement anomaly detection for resource management in a cloud-edge environment is DeepRM_AD and DeepRM_ECO [8]. While they showcase its efficacy and importance, they have used a simplified approach to capturing the state-action relationship of the initial DeepRM framework.

Our research aims to address the existing gap by synthesising the modelling and training efficiencies of DeepRM_plus with the anomaly detection capabilities of DeepRM_AD and DeepRM_ECO. This proposed synthesis has the potential to create a comprehensive framework that not only excels in managing resources across cloud and edge computing efficiently but also in addressing system anomalies robustly. By integrating a CNN and imitation learning with effective anomaly detection, we aspire to establish a new standard for resilience and efficiency in cloud and edge computing resource management, potentially transforming the field. This approach underscores the potential of RL to optimise resource management while ensuring system reliability amidst disruptions.

This research directly addresses these challenges, guided by a central question and specific sub-questions that aim to explore and quantify the benefit of integrating DRL with anomaly detection in cloud-edge computing environments. The principal research question this study aims to answer reads:

To what extent does integrating DRL-based methods with anomaly detection methods enhance resource management efficiency in the cloud-edge continuum?

The research can be further divided into the following sub-questions:

- *What are the impacts of replacing a time-series neural network with a convolutional neural network (CNN) in DRL models on the state observation accuracy and training efficiency in cloud-edge environments?*
- *How does the addition of a CNN influence anomaly detection accuracy within DRL models in cloud-edge environments?*

2 RELATED WORK

This section discusses the relevant topics in developing our fault-tolerant RL agent.

2.1 Cloud-edge computing

The emergence of cloud computing has significantly transformed the IT landscape by making computing resources accessible as a utility. This shift allows outsourcing data centre operations to providers offering centralised computing facilities through a Software as a Service (SaaS) model. This innovation allows applications to scale dynamically in response to user demand, linking costs directly to the level of service consumption [3, 4, 7]. However, the computational resources available at the network edge are limited, presenting challenges for time-sensitive and real-time applications. This shortfall has driven the advent of edge computing [12], distinguished by its reduced latency due to proximity to data sources, which significantly cuts down data transmission times and alleviates bandwidth constraints [10, 31, 33, 34]. This paradigm is particularly advantageous for applications requiring immediate data processing, such as web-based services [43], real-time video analytics [2], and augmented reality (AR) technologies [35]. An integrated approach that synergises the cloud's scalability with edge computing's bandwidth efficiency and low-latency capabilities could significantly enhance the performance and efficiency of diverse applications.

2.2 Job scheduling

The field of job scheduling has a longstanding history, dating back to 1954 when the first mathematical model for scheduling across two and three machines was introduced [20]. This research is motivated by challenges in production scheduling, computer management, and broadly in scenarios where there is a need to allocate limited resources to activities over time [21]. Depending on the specific parameters and goals, job scheduling problems are classified as either NP-complete [37] or NP-hard [36]. These challenges are defined by the task of scheduling a set of jobs J_i for $i = 1, 2, \ldots, n$, on a set of machines M_k for $k = 1, 2, \ldots, m$, where the objective function is often related to the completion times of the jobs, such as job completion time, mean flow time, mean tardiness, and other similar metrics. [15, 44].

In cloud computing, extensive research has been conducted on task scheduling utilising various approaches [5]. An early effort in this domain was by He et al. [18], who proposed a Quality of Service (QoS) guided task scheduling algorithm based on adaptive scheduling heuristics. Owing to the NP-hard nature of job scheduling, most traditional methods employ (meta-)heuristics to find a near-optimal solution within a reasonable timeframe [25]. Commonly used scheduling heuristics include First Come First Serve (FCFS) [6], Shortest Job First (SJF) [11], and Weighted Round Robin (WRR) [38]. Although these strategies often lead to an acceptable near-optimal solution, they struggle to perform well as the complexity of the cloud-edge continuum increases. To better manage uncertainty in cloud environments, intelligent cloud resource management solutions employing DRL have been introduced [42]. Mao et al. [27] proposed DeepRM, utilising DRL to enhance learning efficiency in the given environment, outperforming many popular heuristics. Building on this, Guo et al. [17] introduced DeepRM_plus, which incorporates imitation learning to mimic classical heuristics and employs a refined CNN for improved state space representation. Our work extends these efforts, leveraging DRL for enhanced job scheduling while introducing anomaly detection.

2.3 Anomaly detection

Anomaly detection is the process of identifying data patterns that deviate from the norm or expected behaviour [9]. Anomaly detection is extensively used across various fields, including healthcare, fraud prevention, cyber security, industrial damage assessment, image analysis, sensor networks, robotic actions, and the study of astronomical data [1]. In cloud environments, there has been a surge in the development of techniques for detecting anomalies, driven by the increasing complexity of the cloud, which leads to anomalies occurring more frequently [13, 19]. Much of the research on anomalies in cloud environments concentrates on identifying different kinds of malware and Denial of Service (DoS) attacks. For instance, Pandeeswari and Kumar [29] proposed an Intrusion Detection System designed to identify attacks in cloud environments by utilising a hybrid approach that combines the Fuzzy C-Means clustering algorithm with an Artificial Neural Network (FCM-ANN). Watson et al. [39] introduce an online cloud anomaly detection technique using a one-class Support Vector Machine (SVM) algorithm to detect malware and DoS attacks. Although their results are promising, their research is limited to attacks, neglecting that anomalies in cloud environments can also manifest as outliers unrelated to attacks. Moreover, these anomaly detection algorithms fall short in efficiency for network anomaly detection that necessitates analysis in real-time [14].

Deep learning (DL) has recently become a notable trend, attracting research interest in the area of network anomaly detection. Models that utilise non-linear architectures, such as CNN [30], recurrent neural networks (RNN) [16], and Long Short-Term Memory (LSTM) [26], have demonstrated enhanced learning capabilities and improved accuracy rates in detecting anomalies. Garg et al. [13] propose a hybrid data processing model for network anomaly detection that leverages in a first place grey wolf optimisation (GWO) for feature selection, and then CNN is used to classify network anomalies into normal and anomalous classes. Liu et al. [24] developed a deep anomaly detection framework based on federated learning for time-series data in Industrial IoT, incorporating an attention mechanism with CNN and LSTM (AMCNN-LSTM). The success of CNN in these studies, which outperform established benchmarks, underscores its suitability for anomaly detection. However, while these findings are promising, the studies primarily focus on identifying anomalies without leveraging this information to optimise resource management in the cloud-edge computing continuum.

The study most similar to our work was conducted by Barring [8], who introduced DeepRM_ECO, an online DRL job scheduling algorithm with integrated anomaly detection. In contrast to DeepRM and DeepRM_Plus, this algorithm can allocate jobs to either edge nodes or a single cloud node, effectively simulating a basic edge-cloud environment. Moreover, DeepRM_ECO's reward function is designed to address anomalies while minimising job slowdown.

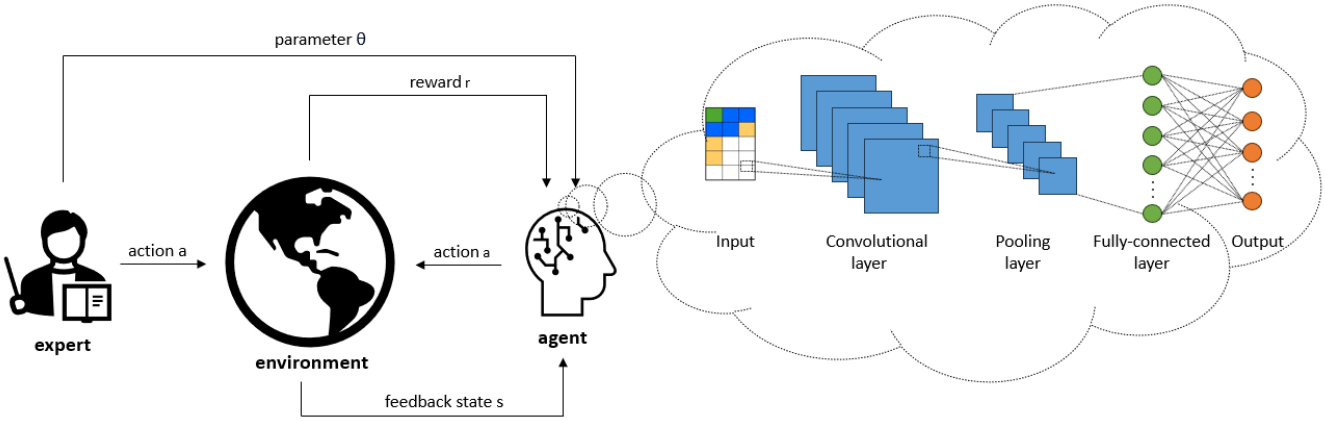

Figure 1: Workflow of DeepRM_2: Imitation learning using SJF heuristics establishes a basic policy, which is then refined by a DRL agent using a CNN. The state space image is used as input for the CNN to optimise policy decisions.

Our objective is to bridge the gap in the existing literature by integrating the capability of DeepRM_ECO to utilise anomaly-related information with the enhanced state space representation offered by DeepRM_Plus through the use of CNN and imitation learning techniques.

3 METHODOLOGY

Our model development will leverage the DeepRM_ECO [8] framework, which is built on top of the original DeepRM [27] framework. This framework simulates a simplified cloud-edge environment in which jobs can either be allocated to a cloud instance with high computing power or two edge instances with limited resources. Our enhanced model, which we call DeepRM_2, builds on this framework by implementing a CNN that is designed to analyse images depicting the current resource utilisation within the cluster to deduce the optimal policy, similar to DeepRM_plus [17]. The aim of implementing a CNN is to capture the current state space more accurately, which should lead to less job slowdown and potentially faster convergence of the agent.

To efficiently narrow down the policy search space before applying DRL, we will initially employ imitation learning to establish a basic strategy similar to DeepRM_ECO. This method of imitation learning falls into the domain of behaviour cloning, where our agent follows an initial policy derived from expert heuristics, specifically SJF. Subsequently, we will refine the policy through DRL with a CNN, continuing until the agent converges on the optimal policy. The overall workflow of DeepRM_2 is illustrated in Figure 1.

Architecture. The architecture of our environment, as depicted in Figure 2, comprises three main components:

- *Backlog*
- *Waiting Queue*
- *Machine 1, 2, and 3*

The backlog, depicted in the top part of the framework diagram, stores information about all jobs beyond the initial M jobs. Jobs are transferred from the backlog to the waiting queue when a job is successfully allocated to a machine. The waiting queue contains jobs prepared for allocation to either machine when the machine is currently occupied with other jobs. Each job, whether normal or anomalous, can be allocated to either an edge or a cloud instance. However, the DRL agent is tasked with identifying these anomalous jobs to migrate them to the cloud instance and process them faster. Machine 1 (M1) and Machine 2 (M2) simulate edge instances with limited resources, while Machine 3 (M3) acts as a cloud instance equipped with more substantial resource capabilities. This setup is chosen to align with the configuration used in DeepRM_ECO [8], facilitating direct comparison.

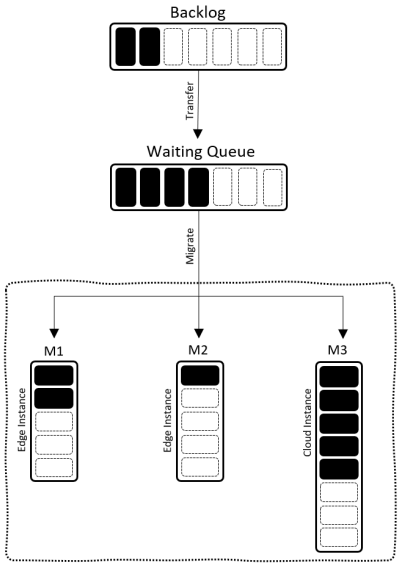

Figure 2: Architecture of DeepRM_2: Jobs move from the backlog to the waiting queue when ready, then to Machines (M1 and M2 as edge instances with limited resources, and M3 as a cloud instance with substantial resources) for processing. Arrows indicate the job flow.

State Space: A comparable resource allocation approach to that of DeepRM_ECO will be employed. However, instead of using an array with a fixed size to represent the jobs that can be held, we will feed the state space information as an image to a deep neural network (DNN), similar to DeepRM_plus. The CNN utilised consists of five layers. The network includes one convolutional layer with 16 kernels, followed by max-pooling. This layer is connected to a fully connected layer, which passes through a 50% dropout layer. The output layer provides the probability of selecting a specific action in the current state. The parameters of the CNN are detailed in Table 5, Appendix C.

Figure 3 depicts an illustrative example of the state space, which visually represents the resource allocation. The state space consists of three distinct components:

- *(Virtual) Data centre cluster* (leftmost part)
- *Waiting queue* (rightmost part)
- *Backlog queue* (bottom part)

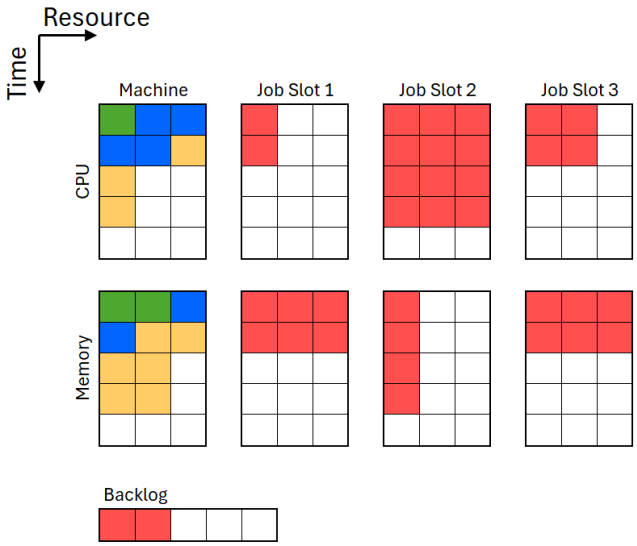

Figure 3: Resource allocation in DeepRM_2: The data centre cluster with three machines, each with two resources. Resource requirements are along the horizontal axis, while time is on the vertical axis, with colours indicating different jobs. The waiting queue (Job Slots 1 to 3) shows jobs awaiting allocation. The backlog queue shows the number of jobs waiting without specifying resource requirements.

The (virtual) data centre cluster in our scenario comprises three machines: M1, M2, and M3, as previously described. Each of these machines possesses two (limited) resources, which can be interpreted as, for example, CPU and memory. The resource requirements are represented along the horizontal axis, while the upcoming job slots (time) are depicted along the vertical axis. The colour variations signify different jobs. For instance, in Figure 3, the blue job is scheduled to utilise two units of CPU and one unit of memory for the next two timesteps. The waiting queue serves as a repository for jobs awaiting allocation. In this queue, the horizontal length signifies the resource demand of a particular job, while the vertical length represents its completion time. For example, consider job slot 1: It necessitates one unit of CPU and three units of memory, lasting for two timesteps. Finally, the backlog queue reflects the number of jobs waiting without specifying the resources required for their completion.

Action Space: We employ the same discrete action space used in the original implementation of DeepRM_ECO. The agent has four possible actions to choose from:

- *Migrate M1*: Migrate job to machine 1
- *Migrate M2*: Migrate job to machine 2
- *Migrate M3*: Migrate job to machine 3
- *MoveOn*: Move on to the next timestep: $t = t + 1$

Reward: DeepRM is specifically designed to minimise mean job slowdown, influencing its reward system. To compare our agent with DeepRM_ECO, we use the same reward function as DeepRM_ECO. This approach incentivises the agent to detect anomalies and allocate them to the cloud instance. The reward function is represented by Equation 1 and its pseudo-code can be found in Algorithm 1, Appendix A:

$$R = \sum_{j \in J} \frac{-1}{T_j} + \sum_{j \in C} \begin{cases} 3, & \text{if } T_j \geq A_{len} \\ -2, & \text{otherwise} \end{cases} \tag{1}$$

Let J represent all jobs in the backlog, waiting queue, and the edge instances (M1 and M2), while C represents the cloud instance M3. For jobs in J, a proportional penalty of $\frac{-1}{T_j}$ is applied, where T_j is the duration of job j. For jobs that have been migrated to C, if the job duration T_j meets or exceeds the anomaly threshold A_{len}, the agent receives a reward of 3. Conversely, if the job is incorrectly identified as anomalous (i.e., T_j is less than A_{len}), a penalty of -2 is applied.

4 IMPLEMENTATION

This section outlines our study's approach, including a description of the dataset used for training and testing the model. Additionally, we detail the benchmark and comparative methods employed to assess the model's effectiveness. Finally, we explain the evaluation method used for the proposed model.

4.1 Problem Formulation

A substantial volume of data is required to train a DRL agent effectively. Given the challenge of acquiring vast datasets, many DRL approaches rely on synthetic data to identify the best policy. Accordingly, our agent will be trained using a synthetic dataset generated by DeepRM_ECO [8], derived from the original DeepRM [27]. While the used dataset adopts a simplified approach to encapsulate the aspects of multi-resource scheduling, it remains a complex scenario, enabling us to examine the efficacy of DRL techniques in cloud computing.

To generate anomalous data within the dataset, we use the data generation module (see Algorithm 2, Appendix A) to create jobs characterised by either unusually long execution times or resource

requirements exceeding those of normal jobs. The generation of these anomalies will be random yet controlled to prevent them from overwhelming the typical job distribution, as this does not typically reflect real-world conditions. Moreover, this approach allows us to experiment with varying levels of anomalous data to observe its impact on the agent's performance.

Data features. Three primary attributes characterise a job generated by DeepRM:

- job_id
- job_duration
- resource profile

The job id, represented by j_i, is a unique identifier for each job. The job duration, denoted by T_j, ranges from 1 to 18t. Jobs that exceed this duration are categorised as anomalous, and their maximum duration is capped at 28t to manage their occurrence.

$$T_{j_{Normal}} = \{T \in \mathbb{N} \mid 1 \leq T \leq 18\}$$
$$T_{j_{Anomalous}} = \{T \in \mathbb{N} \mid 19 \leq T \leq 28\}$$

Furthermore, each job consists of a resource profile r_j that is known upon job arrival and consists of d resource types, such as CPU, memory, or I/O. We fixed the number of resource types to two and operated under the assumption of no preemption and a static allocation profile. This means that r_j must be consistently allocated from the start of execution until the job is complete, mirroring the approach used in DeepRM. The job resource profile is represented as a vector:

$$\vec{r}_j = \left([r1_{j_1}, r2_{j_1}], [r1_{j_2}, r2_{j_2}], \ \ldots, [r1_{j_n}, r2_{j_n}]\right)$$

Moving forward, we will specify what qualifies as normal and what qualifies as anomalous in terms of resource requirements:

$$r_{j_{Normal}} = \{r \in \mathbb{N} \mid 1 \leq r \leq 10\}$$
$$r_{j_{Anomalous}} = \{r \in \mathbb{N} \mid 11 \leq r < 15\}$$

When a job is allocated and executed, it is further enriched with three additional attributes:

- Job enter time, denoted as j_E
- Job start time, denoted as j_S
- Job finish time, denoted as j_F

While these attributes are useful for assessing the agent's efficiency and for calculating various metrics, the agent primarily considers the job duration T_j and its resource profile r_j in its evaluations.

Exploratory Data Analysis. As previously mentioned, we can adjust the quantity of anomalous data generated by DeepRM. For our experiments, we will use different levels of anomalous jobs. In most cases, we will primarily use approximately 10% anomalous jobs, with roughly 5% of the jobs having an anomalous job duration and another 5% having an anomalous resource profile, meaning that one of the resources is anomalous.

As shown in Figure 4, DeepRM generates jobs with both normal and anomalous durations, distinguished in the graph by different colours. The normal jobs are further divided into small and big categories. Small jobs are defined as $[1, T_{j_{max}}/5] = [1, 3.6]$ and big jobs are defined as $[T_{j_{max}} * 2/3, T_{j_{max}}] = [12, 18]$, where $T_{j_{max}}$ denotes the maximum normal job duration. It is noted that approximately 70% of the jobs are small, 25% are big, and the remaining 5% are of anomalous duration. The pattern of resource consumption follows a similar pattern and is detailed in Figure 10, Appendix B.

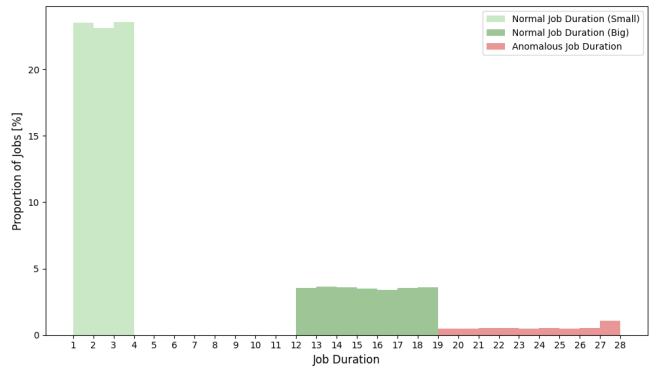

Figure 4: Relative distribution of job duration with an anomaly rate of 0.05

The relationship between the two resources is depicted in Figure 5, showing that each job always features one dominant resource and one inferior resource. The dominant resource is either categorised as big ([5, 10]) or anomalous ([11, 14]), whereas the inferior resource is consistently small ([1, 2]). It is random which resource becomes dominant, with each having a 50% chance of being the dominant one. Notably, only 2.5% of resources are anomalous, indicating that about 5% of resource profiles fall into the anomalous category.

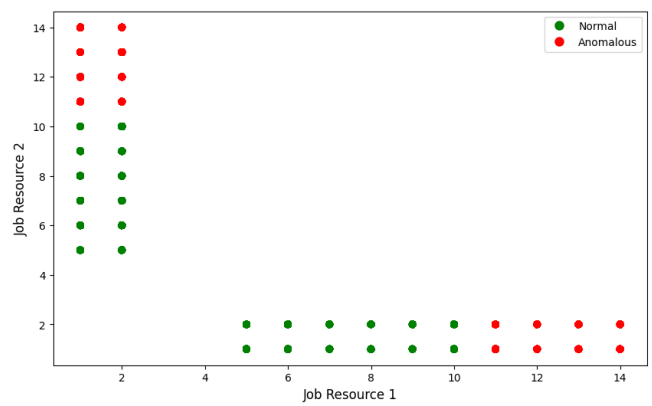

Figure 5: Relationship between job resource 1 and job resource 2

4.2 Baseline

To assess the performance of our agent, we implement two baseline models: the state-of-the-art DeepRM_ECO and SJF heuristics. Initially, we compare the mean job slowdown and cumulative reward over 1000 epochs across these models to evaluate our agent's relative performance during training, with SJF heuristics serving

as a basic threshold. Following this, we test all models with unseen data across different simulation lengths, ranging from 10 to 200 in increments of 10, with 20 examples per increment, and compare their mean job slowdown. We employ the Mann-Whitney U test to evaluate the significance of the results. Furthermore, we assess the anomaly detection capabilities of our models with different anomaly rates, 0.10, 0.25, and 0.50, using the two-proportion z-test to check for significance.

4.3 Evaluation metrics

To assess the effectiveness of our proposed model, we utilise several key metrics: Mean job slowdown (S_j), mean and maximum cumulative reward, along with recall, precision, F1 score, and accuracy for evaluating the anomaly detection component. The mathematical formulations of these metrics are detailed in the equations below, except the reward function, which is shown in Equation 1:

$$S_j = \frac{C_j}{T_j}$$
$$\overline{R} = \frac{\sum R}{x}$$
$$R_{max} = max(R)$$
$$Recall = \frac{TP}{TP + FN}$$
$$Precision = \frac{TP}{TP + FP}$$
$$F1Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$
$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

In the above equations, the parameters TP, TN, FP and FN refer to *True Positive*, *True Negative*, *False Positive*, and *False Negative*, respectively. C_j is the *completion time* of job j, and T_j is the *(ideal) duration* of the job; note that $S_j \geq 1$. Moreover, to assess whether the CNN enhances training speed, we will additionally measure the time it takes for the agent to converge. This evaluation will involve tracking both the number of epochs and time per epoch.

Experimental setup: To optimise the training time, the experiments will be conducted on an instance created on the Snellius supercomputer. It leverages GPU, which provides a node with NVIDIA A100 GPUs (40GB). All parameters used were tracked for reproducibility.

5 RESULTS

In this section, we present the findings from our job scheduling experiments. Our contribution involves using a DRL agent with an incorporated CNN model to capture the state space in an anomalous environment. The model detects and classifies anomalous incoming tasks, subsequently determining whether to schedule them to a (virtual) cloud or edge instance.

5.1 Mean Job Slowdown

Training results. We first observe the mean slowdown over 1000 epochs, as shown in Figure 6. It is apparent that our implementation outperforms the DeepRM_ECO baseline in terms of convergence. Although both models achieve a similar mean job slowdown of around 1.22 after 1000 epochs, DeepRM_2 converged already after approximately 500 epochs to the optimal policy, while DeepRM_ECO has not fully converged after 1000 epochs. While DeepRM_2 converged with fewer epochs, the training time needed per epoch increased by 230.45% from an average of 2.66s to 8.79s. Furthermore, unlike DeepRM_ECO, our implementation of DeepRM_2 exhibits occasional spikes throughout the training process. The SJF heuristics mean job slowdown on the training data is 3.59, which DeepRM_2 surpasses after approximately 50 epochs.

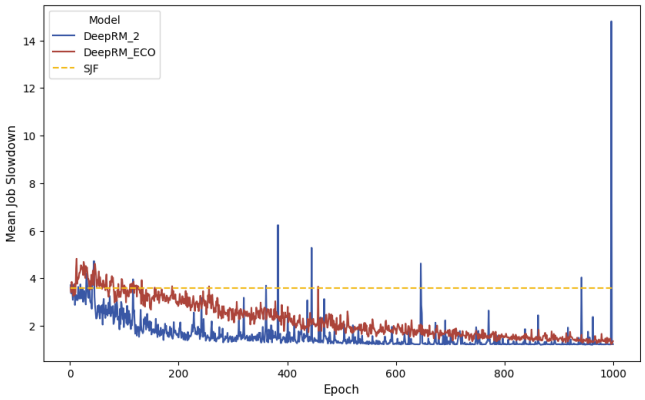

Figure 6: Mean job slowdown over 1000 epochs with an anomaly rate of 0.1, comparing different models: DeepRM_2 (blue), DeepRM_ECO (red), and SJF heuristics (yellow dotted)

The mean and maximum cumulative rewards for both DeepRM algorithms are displayed in Figure 7. Both implementations see an increase in cumulative rewards with each epoch. After 1000 epochs, DeepRM_2 achieves a mean cumulative reward of approximately 3.71, while DeepRM_ECO achieves about 3.08. Dispersion between mean and maximum cumulative reward indicates that further training is needed. DeepRM_2's mean and maximum cumulative rewards converge closely after just 600 epochs, whereas DeepRM_ECO still exhibits slight dispersion even after 1000 epochs, indicating that additional training might be necessary for optimal performance.

To ensure a fair comparison of the models, we retrained DeepRM_ECO for 3000 epochs, achieving convergence. The mean job slowdown and cumulative reward from this training process are illustrated in Figure 11 and 12, Appendix C, respectively.

Test results. After completing the training process, we assessed all models using unseen data across various simulation lengths. DeepRM_2 consistently achieved the lowest mean job slowdown for simulation lengths exceeding 20. Initially, SJF performed well ($simu_len < 50$), but its mean job slowdown increased significantly as the simulation length grew, in contrast to the slight increase observed with DeepRM_2. DeepRM_ECO exhibited notable fluctuations in mean job slowdown, with peaks reaching approximately

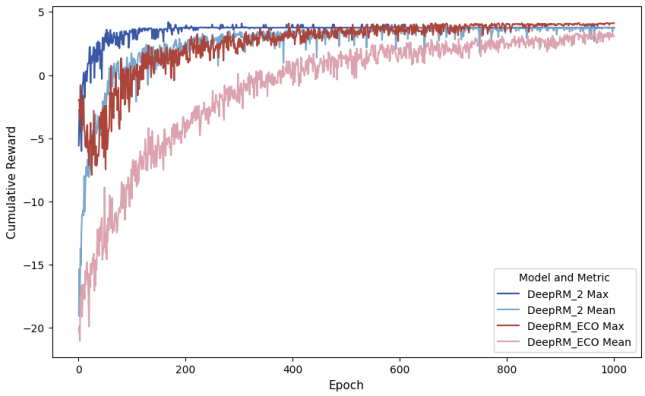

Figure 7: Mean and maximum cumulative reward over 1000 epochs with an anomaly rate of 0.1, comparing different models: DeepRM_2 (blue) and DeepRM_ECO (red)

80. However, there were instances where DeepRM_ECO's performance closely resembled that of DeepRM_2, indicating potential under specific conditions. On average, across all simulation lengths, DeepRM_2 reduced mean job slowdown by 50.29% compared to DeepRM_ECO and 55.71% compared to SJF.

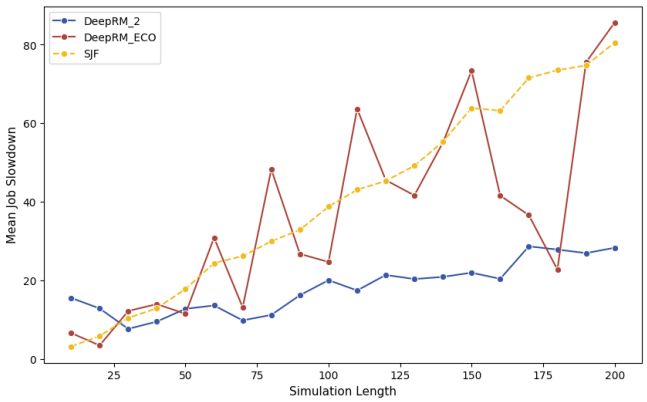

Figure 8: Mean job slowdown for simulation length ranging from 10 to 200 with an anomaly rate of 0.1, comparing different models: DeepRM_2 (blue), DeepRM_ECO (red), and SJF heuristics (yellow dotted)

To determine if DeepRM_2 performs significantly better than the baseline models, we conducted the Mann-Whitney U test comparing DeepRM_2 with each of the other two models, using a sample size of 20 data points for each group, representing 20 different simulation lengths. As shown in Table 1, the results are significant for DeepRM_2 vs DeepRM_ECO (P-value = 0.0167, U = 111) and for DeepRM_2 vs SJF (P-value = 0.0028, U = 89). These results indicate that DeepRM_2 significantly outperforms both DeepRM_ECO and SJF regarding mean job slowdown across different simulation lengths. The table also reports the effect sizes using rank-biserial correlation.

Table 1: Mann-Whitney U test results for mean job slowdown across different simulation lengths, comparing DeepRM_2 against DeepRM_ECO and SJF

Comparison	U	P-value	Effect Size
DeepRM_2 vs DeepRM_ECO	111	0.0167	0.445
DeepRM_2 vs SJF	89	0.0028	0.555

5.2 Anomaly Detection

To evaluate the anomaly detection capabilities of DeepRM_2, we compared its performance to that of DeepRM_ECO. Table 2 presents the performance metrics for DeepRM_2, while Table 3 displays the performance metrics for DeepRM_ECO. Each model was tested with anomaly rates of 0.10, 0.25, and 0.50. For both models, recall, precision, and F1 score tended to improve with higher anomaly rates, although accuracy showed a slight decline as the anomaly rate increased. Overall, DeepRM_2 consistently outperformed DeepRM_ECO across all metrics, demonstrating more reliable and effective anomaly detection capabilities. DeepRM_2's superior performance was evident in its ability to maintain higher precision and F1 scores, particularly at lower anomaly rates, making it a more robust choice for anomaly detection tasks.

Table 2: Anomaly detection evaluation for DeepRM_2 for anomaly rate 0.10, 0.25, and 0.50

Anomaly Rate	Recall	Precision	F1 Score	Accuracy
0.10	0.738	0.422	0.537	0.880
0.25	0.753	0.577	0.654	0.824
0.50	0.799	0.722	0.759	0.781

Table 3: Anomaly detection evaluation for DeepRM_ECO for anomaly rate 0.10, 0.25, and 0.50

Anomaly Rate	Recall	Precision	F1 Score	Accuracy
0.10	0.494	0.143	0.221	0.761
0.25	0.583	0.347	0.435	0.737
0.50	0.647	0.583	0.614	0.701

Figure 9 displays the corresponding confusion matrices to complement the metrics discussed earlier. At an anomaly rate of 0.10, DeepRM_2 predominantly predicts jobs as normal, with over 50% of the predicted anomalous jobs being incorrectly classified. As the anomaly rate increases, DeepRM_2 predicts more jobs as anomalous. At an anomaly rate of 0.50, it predicts nearly equal numbers of anomalous and normal jobs, with a higher tendency to incorrectly classify normal jobs as anomalous rather than the reverse. DeepRM_ECO shows similar behaviour but with a higher rate of false positives (0.268 vs 0.233 at an anomaly rate of 0.50), incorrectly classifying a greater percentage of normal jobs as anomalous. Additionally, DeepRM_ECO has a higher false negative rate (0.353 vs 0.201 at an anomaly rate of 0.50), further indicating challenges in correctly identifying anomalous jobs.

To evaluate the anomaly classification performance of DeepRM_2 and DeepRM_ECO across the used anomaly rates, we conducted

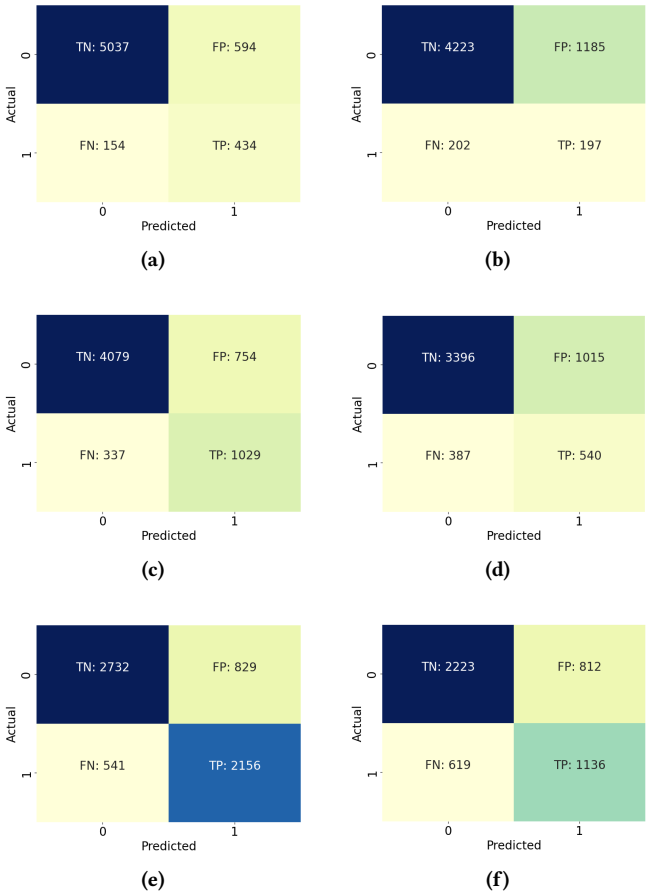

Figure 9: Comparison of confusion matrices for DeepRM_2 (left) and DeepRM_ECO (right) at different anomaly rates: (a) DeepRM_2: anomaly rate 0.10, (b) DeepRM_ECO: anomaly rate 0.10, (c) DeepRM_2: anomaly rate 0.25, (d) DeepRM_ECO: anomaly rate 0.25, (e) DeepRM_2: anomaly rate 0.50, (f) DeepRM_ECO: anomaly rate 0.50.

two-proportion z-tests on their respective proportions of correct classifications. The test results for anomaly rates of 0.10, 0.25, and 0.50 are summarised in Table 4. The results indicate that DeepRM_2 performs significantly better than DeepRM_ECO across all anomaly rates. Specifically, the differences are significant for anomaly rates of 0.10 (P-value = 7.60×10^{-65}, Z-statistic = 17.00), 0.25 (P-value = 1.74×10^{-29}, Z-statistic = 11.28), and 0.50 (P-value = 1.20×10^{-21}, Z-statistic = 9.56).

Table 4: Two-Proportion Z-Test, comparing DeepRM_2 and DeepRM_ECO with anomaly rates 0.10, 0.25, and 0.50

Anomaly Rate	Z-statistic	P-value
0.10	17.00	7.60×10^{-65}
0.25	11.28	1.74×10^{-29}
0.50	9.56	1.20×10^{-21}

6 DISCUSSION

This section compares the outcome of our research with existing literature mentioned in Section 2, followed by the limitations that our model and environment are constrained to.

6.1 Comparative Study

Similar to the approach of Guo et al. [17] with DeepRM_Plus, we implemented a CNN to enhance the original DeepRM framework developed by Mao et al. [27]. However, unlike their approach, we integrated an anomaly detection module, introduced in DeepRM_ECO by Barring [8]. These studies collectively concluded that using DRL outperforms heuristics for multiresource scheduling problems. Furthermore, DeepRM_Plus demonstrated that incorporating a CNN enables the agent to learn an optimal policy more efficiently. Our agent, DeepRM_2, similarly required fewer epochs to converge. However, the training duration per epoch increased by 230.45% compared to DeepRM_ECO due to the more complex state space. When considering the total number of epochs trained, both models required approximately the same overall training duration.

DeepRM_2 also outperformed DeepRM_ECO on unseen data regarding mean job slowdown, showcasing better generalisability. By conducting Mann-Whitney U tests, we found that DeepRM_2 significantly outperforms both DeepRM_ECO and SJF in reducing mean job slowdown across different simulation lengths. These findings demonstrate that DeepRM_2 not only excels under training conditions but also maintains its efficiency over various simulation lengths, underscoring its robustness and adaptability in different scenarios. This enhanced performance can be attributed to incorporating of a CNN, which inputs an image of the state space rather than an array. This approach allows for better feature extraction and pattern recognition, enabling the agent to learn and generalise more effectively.

Regarding anomaly detection, our experiments revealed that both models exhibit lower precision and F1 scores at anomaly rates of 0.10 and 0.25 compared to 0.50. This lack of performance with a low anomaly rate can be attributed to the design of the reward function. The agent learned to migrate a normal job to an empty cloud node (M3) or when the edge nodes (M1 and M2) were full, resulting in a short-term penalty less severe than the penalty for increasing job slowdown. To enhance precision and F1 scores, adjustments to the reward function to impose stricter penalties on incorrect classifications could be considered. However, predicting a normal job as anomalous is not necessarily negative, as it can reduce job slowdown. A more critical issue arises when an anomalous job is misclassified as normal, leading to penalties and inappropriate assignment to an edge node. Despite this, in situations where many anomalous jobs arrive consecutively, allocating anomalous jobs to edge machines might be more effective rather than leaving them idle, preventing the cloud machine from becoming overloaded.

Furthermore, our statistical analysis demonstrated that DeepRM_2 significantly outperforms DeepRM_ECO in classifying anomalies across all tested anomaly rates. This improved performance indicates that introducing a non-linear CNN in the network architecture enhanced the agent's ability to classify incoming jobs more accurately, thereby reducing in mean job slowdown. Consequently, DeepRM_2 proves to be a more robust and effective solution for

managing job scheduling and anomaly detection in dynamic cloud-edge environments.

6.2 Limitations

While this study yields promising results in the domain of DRL and anomaly detection, several limitations must be acknowledged.

The foremost limitation is the reliance on a synthetic dataset rather than real-world data. Although the data generation module from DeepRM is efficient in creating specific jobs, future research should employ real-world datasets to validate the agent's performance. In practical scenarios, the job length and resource profile are often unknown in advance, significantly complicating the job scheduling problem. This issue can be addressed using a partially observable Markov decision process (POMDP), which, instead of determining the environment's resulting state s', receives an observation $o \sim O(\cdot|s, a)$ [28]. While this approach would most likely result in a slight decline in performance, SJF is also expected to perform worse as it cannot identify the shortest jobs. Other heuristics like WRR or FCFS might become more valuable in such cases.

Another limitation of this study is the short simulation length ($simu_len = 10$) on which the agent was trained. This constraint was imposed due to computational limitations, as the memory requirements for training DeepRM_2 increased substantially with longer simulation lengths. Ideally, the simulation length should be extended, as our tests involved simulation lengths up to 200. While our agent still outperformed any heuristic in these scenarios, it might have demonstrated even greater efficacy if trained on such extended simulation lengths. DeepRM_ECO was particularly affected by the short simulation length, as it failed to generalise well to longer simulation lengths, resulting in significant fluctuations in mean job slowdown. However, we used the same simulation length for both to ensure a fair comparison between the two agents.

Initially, we implemented a more complex CNN architecture with two convolutional layers, having 32 and 64 filters respectively. This design increased the receptive field, allowing the network to capture more spatial context and detailed features beyond low-level patterns. However, we ultimately opted for a simpler network architecture due to the increasing computational cost during the agent's training process.

Finally, the reward function used in this study limits the achievable total mean job slowdown, as it integrates the anomaly detection module. If the sole goal is to minimise mean job slowdown, the anomaly detection component should be excluded from the reward function to optimise this metric.

7 CONCLUSION

In this study, we demonstrated that multi-resource job scheduling problems can be improved by more accurately capturing the state space using a convolutional neural network (CNN) and incorporating incoming anomalies as valuable information for the agent. Building on the DeepRM_ECO framework, we developed DeepRM_2, a new deep reinforcement learning (DRL) model that processes the current state space as an image input to a CNN.

Our findings indicate that the proposed model outperforms both state-of-the-art models and conventional heuristics in a synthetically generated environment. DeepRM_2 showed improved training efficiency by converging in fewer epochs, although each epoch required a longer duration, and some oscillations were observed during the training process. The model also excelled in simulations of varying simulation lengths, having overall 50.29% and 55.71% less mean job slowdown than DeepRM_ECO and SJF, respectively. The significance of the reduction in mean job slowdown was confirmed using the Mann-Whitney U test. Additionally, DeepRM_2 proved to be more accurate in detecting anomalies, as validated by the two-proportion z-test.

However, a notable drawback of DeepRM_2 is the increased memory utilisation, which imposes certain limitations. The most significant limitation is that our agent was trained and tested solely in a synthetic environment with short simulation lengths, which may affect the generalisability of the results. It remains to be seen if the findings hold in real-world scenarios.

Future research could optimise the network and train it with longer simulation lengths. Additionally, exploring an actor-critic reinforcement learning approach might be beneficial, as it can reduce the variance of the policy gradient by utilising the value function, potentially requiring fewer samples to achieve convergence.

REFERENCES

[1] Mohiuddin Ahmed, Abdun Naser Mahmood, and Jiankun Hu. 2016. A survey of network anomaly detection techniques. *Journal of Network and Computer Applications* 60 (2016), 19–31. Publisher: Elsevier.

[2] Ganesh Ananthanarayanan, Paramvir Bahl, Peter Bodík, Krishna Chintalapudi, Matthai Philipose, Lenin Ravindranath, and Sudipta Sinha. 2017. Real-time video analytics: The killer app for edge computing. *computer* 50, 10 (2017), 58–67. Publisher: IEEE.

[3] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D Joseph, Randy Katz, Andy Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, Ion Stoica, and others. 2010. A view of cloud computing. *Commun. ACM* 53, 4 (2010), 50–58. Publisher: ACM New York, NY, USA.

[4] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D Joseph, Randy H Katz, Andrew Konwinski, Gunho Lee, David A Patterson, Ariel Rabkin, Ion Stoica, and others. 2009. *Above the clouds: A berkeley view of cloud computing*. Technical Report. Technical Report UCB/EECS-2009-28, EECS Department, University of California ....

[5] AR Arunarani, Dhanabalachandran Manjula, and Vijayan Sugumaran. 2019. Task scheduling techniques in cloud computing: A literature survey. *Future Generation Computer Systems* 91 (2019), 407–415. Publisher: Elsevier.

[6] Rajkumar Buyya, Rajiv Ranjan, and Rodrigo N Calheiros. 2009. Modeling and simulation of scalable Cloud computing environments and the CloudSim toolkit: Challenges and opportunities. In *2009 international conference on high performance computing & simulation*. IEEE, 1–11.

[7] Rajkumar Buyya, Chee Shin Yeo, Srikumar Venugopal, James Broberg, and Ivona Brandic. 2009. Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility. *Future Generation computer systems* 25, 6 (2009), 599–616. Publisher: Elsevier.

[8] Nils Bärring. 2023. *Learning-Based Anomaly-Tolerant Resource Management in Cloud-Edge Continuum*. Ph. D. Dissertation. University of Amsterdam, Amsterdam, Netherlands.

[9] Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: A survey. *ACM computing surveys (CSUR)* 41, 3 (2009), 1–58. Publisher: ACM New York, NY, USA.

[10] Jiasi Chen and Xukan Ran. 2019. Deep learning with edge computing: A review. *Proc. IEEE* 107, 8 (2019), 1655–1674. Publisher: IEEE.

[11] Archana Ganapathi, Yanpei Chen, Armando Fox, Randy Katz, and David Patterson. 2010. Statistics-driven workload modeling for the cloud. In *2010 IEEE 26th International Conference on Data Engineering Workshops (ICDEW 2010)*. IEEE, 87–92.

[12] Pedro Garcia Lopez, Alberto Montresor, Dick Epema, Anwitaman Datta, Teruo Higashino, Adriana Iamnitchi, Marinho Barcellos, Pascal Felber, and Etienne Riviere. 2015. Edge-centric Computing: Vision and Challenges. *ACM SIGCOMM Computer Communication Review* 45, 5 (Sept. 2015), 37–42. https://doi.org/10.

1145/2831347.2831354
[13] Sahil Garg, Kuljeet Kaur, Neeraj Kumar, Georges Kaddoum, Albert Y Zomaya, and Rajiv Ranjan. 2019. A hybrid deep learning-based model for anomaly detection in cloud datacenter networks. *IEEE Transactions on Network and Service Management* 16, 3 (2019), 924–935. Publisher: IEEE.
[14] Sahil Garg, Amritpal Singh, Shalini Batra, Neeraj Kumar, and Laurence T Yang. 2018. UAV-empowered edge computing environment for cyber-threat detection in smart vehicles. *IEEE network* 32, 3 (2018), 42–51. Publisher: IEEE.
[15] Faruk Geyik and Ismail Hakki Cedimoglu. 2004. The strategies and parameters of tabu search for job-shop scheduling. *Journal of intelligent manufacturing* 15 (2004), 439–448. Publisher: Springer.
[16] Jonathan Goh, Sridhar Adepu, Marcus Tan, and Zi Shan Lee. 2017. Anomaly detection in cyber physical systems using recurrent neural networks. In *2017 IEEE 18th International Symposium on High Assurance Systems Engineering (HASE)*. IEEE, 140–145.
[17] Wenxia Guo, Wenhong Tian, Yufei Ye, Lingxiao Xu, and Kui Wu. 2020. Cloud resource scheduling with deep reinforcement learning and imitation learning. *IEEE Internet of Things Journal* 8, 5 (2020), 3576–3586. Publisher: IEEE.
[18] XiaoShan He, XianHe Sun, and Gregor Von Laszewski. 2003. QoS guided min-min heuristic for grid task scheduling. *Journal of computer science and technology* 18, 4 (2003), 442–451. Publisher: Springer.
[19] Jordan Hochenbaum, Owen S Vallis, and Arun Kejariwal. 2017. Automatic anomaly detection in the cloud via statistical learning. *arXiv preprint arXiv:1704.07706* (2017).
[20] Selmer Martin Johnson. 1954. Optimal two-and three-stage production schedules with setup times included. *Naval research logistics quarterly* 1, 1 (1954), 61–68. Publisher: Wiley Online Library.
[21] Eugene L Lawler, Jan Karel Lenstra, Alexander HG Rinnooy Kan, and David B Shmoys. 1993. Sequencing and scheduling: Algorithms and complexity. *Handbooks in operations research and management science* 4 (1993), 445–522. Publisher: Elsevier.
[22] Dhinesh Babu LD and P Venkata Krishna. 2013. Honey bee behavior inspired load balancing of tasks in cloud computing environments. *Applied soft computing* 13, 5 (2013), 2292–2303. Publisher: Elsevier.
[23] Li Lin, Xiaofei Liao, Hai Jin, and Peng Li. 2019. Computation offloading toward edge computing. *Proc. IEEE* 107, 8 (2019), 1584–1607. Publisher: IEEE.
[24] Yi Liu, Sahil Garg, Jiangtian Nie, Yang Zhang, Zehui Xiong, Jiawen Kang, and M Shamim Hossain. 2020. Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach. *IEEE Internet of Things Journal* 8, 8 (2020), 6348–6358. Publisher: IEEE.
[25] Yongkui Liu, Lihui Wang, Xi Vincent Wang, Xun Xu, and Lin Zhang. 2019. Scheduling in cloud manufacturing: state-of-the-art and research challenges. *International Journal of Production Research* 57, 15-16 (2019), 4854–4879. Publisher: Taylor & Francis.
[26] Pankaj Malhotra, Anusha Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. 2016. LSTM-based encoder-decoder for multi-sensor anomaly detection. *arXiv preprint arXiv:1607.00148* (2016).
[27] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. 2016. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM workshop on hot topics in networks*. 50–56.
[28] Frans A Oliehoek, Christopher Amato, and others. 2016. *A concise introduction to decentralized POMDPs*. Vol. 1. Springer.
[29] N Pandeeswari and Ganesh Kumar. 2016. Anomaly detection system in cloud environment using fuzzy clustering based ANN. *Mobile Networks and Applications* 21 (2016), 494–505. Publisher: Springer.
[30] Hansheng Ren, Bixiong Xu, Yujing Wang, Chao Yi, Congrui Huang, Xiaoyu Kou, Tony Xing, Mao Yang, Jie Tong, and Qi Zhang. 2019. Time-series anomaly detection service at microsoft. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 3009–3017.
[31] Mahadev Satyanarayanan. 2017. The emergence of edge computing. *Computer* 50, 1 (2017), 30–39. Publisher: IEEE.
[32] Shuran Sheng, Peng Chen, Zhimin Chen, Lenan Wu, and Yuxuan Yao. 2021. Deep reinforcement learning-based task scheduling in iot edge computing. *Sensors* 21, 5 (2021), 1666. Publisher: MDPI.
[33] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. Edge computing: Vision and challenges. *IEEE internet of things journal* 3, 5 (2016), 637–646. Publisher: Ieee.
[34] Weisong Shi, George Pallis, and Zhiwei Xu. 2019. Edge computing [scanning the issue]. *Proc. IEEE* 107, 8 (2019), 1474–1481. Publisher: IEEE.
[35] Yushan Siriwardhana, Pawani Porambage, Madhusanka Liyanage, and Mika Ylianttila. 2021. A survey on mobile augmented reality with 5G mobile edge computing: Architectures, applications, and technical aspects. *IEEE Communications Surveys & Tutorials* 23, 2 (2021), 1160–1192. Publisher: IEEE.
[36] Ken W Tindell, Alan Burns, and Andy J. Wellings. 1992. Allocating hard real-time tasks: an NP-hard problem made easy. *Real-Time Systems* 4, 2 (1992), 145–165. Publisher: Springer.
[37] Jeffrey D. Ullman. 1975. NP-complete scheduling problems. *Journal of Computer and System sciences* 10, 3 (1975), 384–393. Publisher: Academic Press.
[38] Weikun Wang and Giuliano Casale. 2014. Evaluating weighted round robin load balancing for cloud web services. In *2014 16th international symposium on symbolic and numeric algorithms for scientific computing*. IEEE, 393–400.
[39] Michael R Watson, Angelos K Marnerides, Andreas Mauthe, David Hutchison, and others. 2015. Malware detection in cloud computing infrastructures. *IEEE Transactions on Dependable and Secure Computing* 13, 2 (2015), 192–205. Publisher: IEEE.
[40] Yufei Ye, Xiaoqin Ren, Jin Wang, Lingxiao Xu, Wenxia Guo, Wenqiang Huang, and Wenhong Tian. 2018. A new approach for resource scheduling with deep reinforcement learning. *arXiv preprint arXiv:1806.08122* (2018).
[41] Deze Zeng, Lin Gu, Shengli Pan, Jingjing Cai, and Song Guo. 2019. Resource management at the network edge: A deep reinforcement learning approach. *IEEE Network* 33, 3 (2019), 26–33. Publisher: IEEE.
[42] Yu Zhang, Jianguo Yao, and Haibing Guan. 2017. Intelligent cloud resource management with deep reinforcement learning. *IEEE Cloud Computing* 4, 6 (2017), 60–69. Publisher: IEEE.
[43] Jiang Zhu, Douglas S Chan, Mythili Suryanarayana Prabhu, Preethi Natarajan, Hao Hu, and Flavio Bonomi. 2013. Improving web sites performance using edge servers in fog computing architecture. In *2013 IEEE Seventh International Symposium on Service-Oriented System Engineering*. IEEE, 320–323.
[44] Banu Çaliş and Serol Bulkan. 2015. A research survey: review of AI solution strategies of job shop scheduling problem. *Journal of Intelligent Manufacturing* 26 (2015), 961–973. Publisher: Springer.

Appendix A PSEUDO CODE

Algorithm 1: Calculate Reward for Jobs

Input: None
Output: Total reward (*output*

```
Function get_reward():
    reward ← 0;
    for each job j in machine.running_job do
        reward ← reward + delay_penalty / j.len;
    end
    for each job j in machine.running_job2 do
        reward ← reward + delay_penalty / j.len;
    end
    for each job j in machine.running_job3 do
        if j.len ≥ anomalous_job_len_lower_bound then
            reward ← reward + (delay_penalty × −3);
        else
            reward ← reward + (delay_penalty × 2);
        end
    end
    for each job j in job_slot1.slot do
        if j ≠ None then
            reward ← reward + hold_penalty / j.len;
        end
    end
    for each job j in job_backlog.backlog do
        if j ≠ None then
            reward ← reward + dismiss_penalty / j.len;
        end
    end
    return reward;
```

Algorithm 2: Generate Normal and Anomalous Jobs

Input: None
Output: Job length (*nw_len*) and job size (*nw_size*)

```
Function generate_job():
    num_res ← 2;
    max_nw_size ← 10;
    job_len ← 18;
    anomalous_job_len_upper_bound ← 29;
    anomalous_job_len_lower_bound ← 19;
    anomalous_job_len_middle_bound ← 23;
    anomalous_resource_lower ← 11;
    anomalous_resource_upper ← 15;
    job_small_chance ← 0.75;
    anomalous_job_rate ← 0.25;
    job_len_big_lower ← job_len × 2/3;
    job_len_big_upper ← job_len;
    job_len_small_lower ← 1;
    job_len_small_upper ← job_len/5;
    dominant_res_lower ← max_nw_size/2;
    dominant_res_upper ← max_nw_size;
    other_res_lower ← 1;
    other_res_upper ← max_nw_size/5;

    random_val ← Random float between 0 and 1;
    if random_val < job_small_chance and random_val > anomalous_job_rate then
        nw_len ← Random integer between job_len_small_lower and job_len_small_upper + 1;
    else
        if random_val < anomalous_job_rate then
            nw_len ← Random integer between anomalous_job_len_lower and anomalous_job_len_upper;
        else
            nw_len ← Random integer between job_len_big_lower and job_len_big_upper + 1;
        end
    end
    nw_size ← Array of zeros with length num_res;
    dominant_res ← Random integer between 0 and num_res;
    random_res ← Random float between 0 and 1;
    for each resource i in range(num_res) do
        if i == dominant_res then
            if random_res > anomalous_job_rate then
                nw_size[i] ← Random integer between dominant_res_lower and dominant_res_upper + 1;
            else
                nw_size[i] ← Random integer between anomalous_resource_lower and anomalous_resource_upper;
            end
        else
            nw_size[i] ← Random integer between other_res_lower and other_res_upper + 1;
        end
    end
    return nw_len, nw_size;
```

Appendix B DATA SET

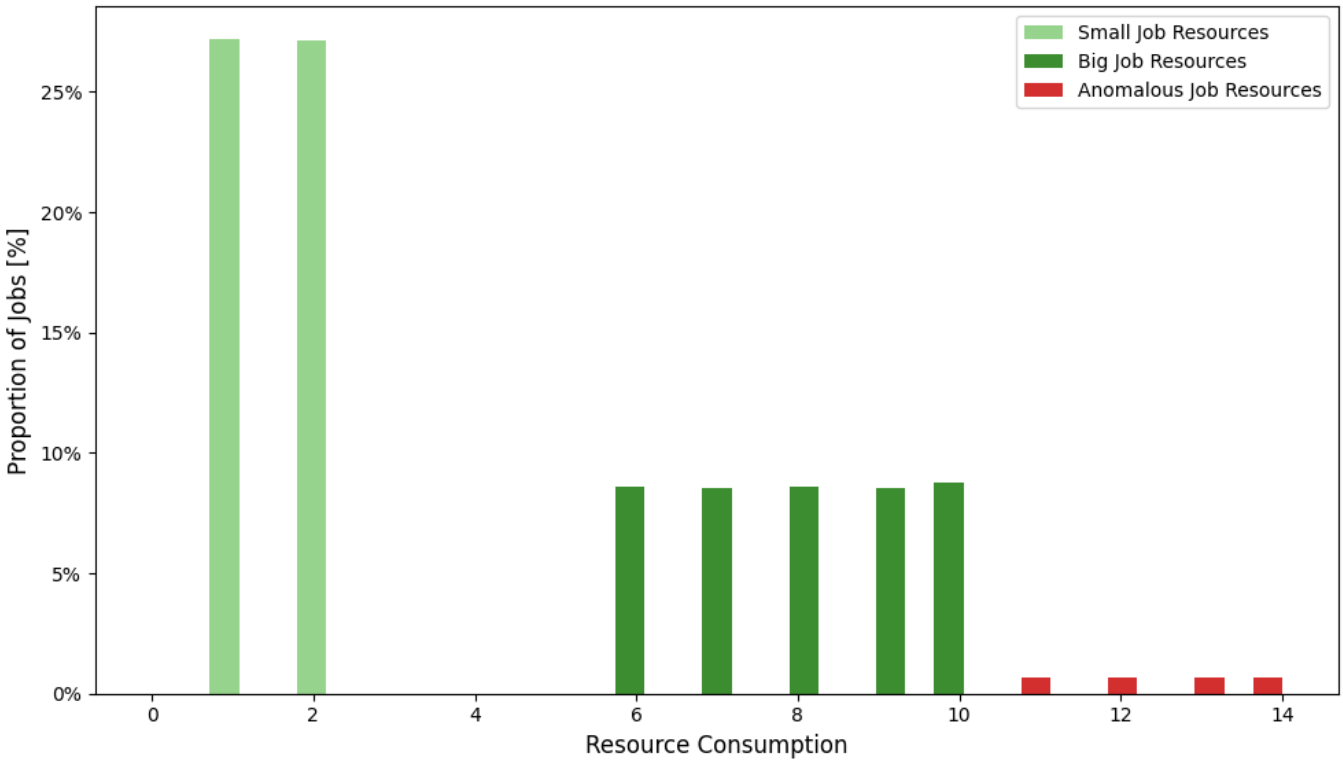

Figure 10: Relative distribution of resource consumption with an anomaly rate of 0.05

Appendix C MODEL & RESULTS

Table 5: Parameters of CNN

Input	Layer	Filter	Filter Num	Activation Function	Output
(30, 225, 1)	Conv1	(3, 3, 1)	16	ReLU	(28, 223, 16)
(28, 223, 16)	MaxPool1	(2, 2)	-	-	(14, 111, 16)
(14, 111, 16)	Dense	-	-	ReLU	64
64	Dropout	-	-	-	64
64	Output	-	-	Softmax	16

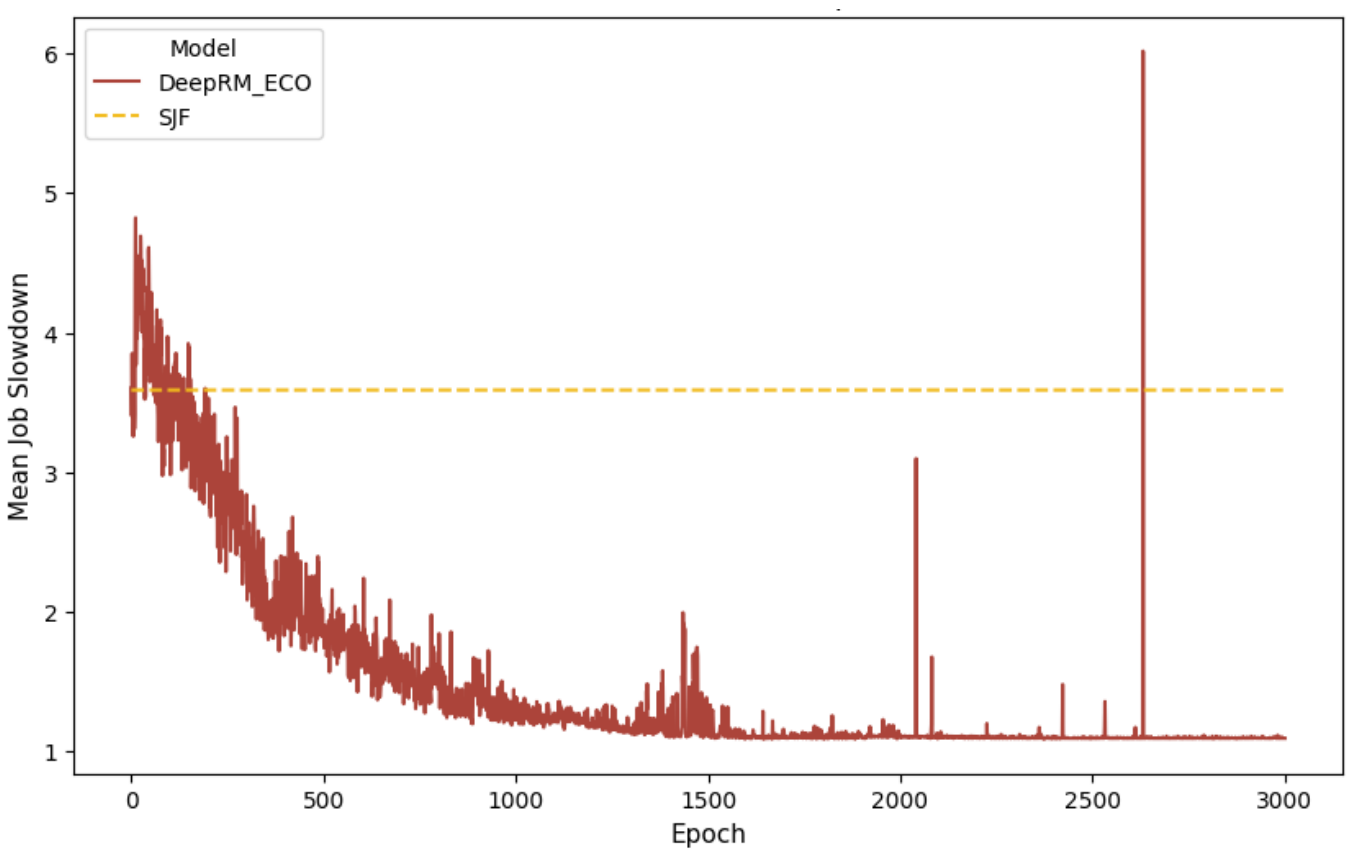

Figure 11: Mean job slowdown over 3000 epochs with an anomaly rate of 0.1, comparing different models: DeepRM_ECO (red), and SJF heuristics (yellow dotted)

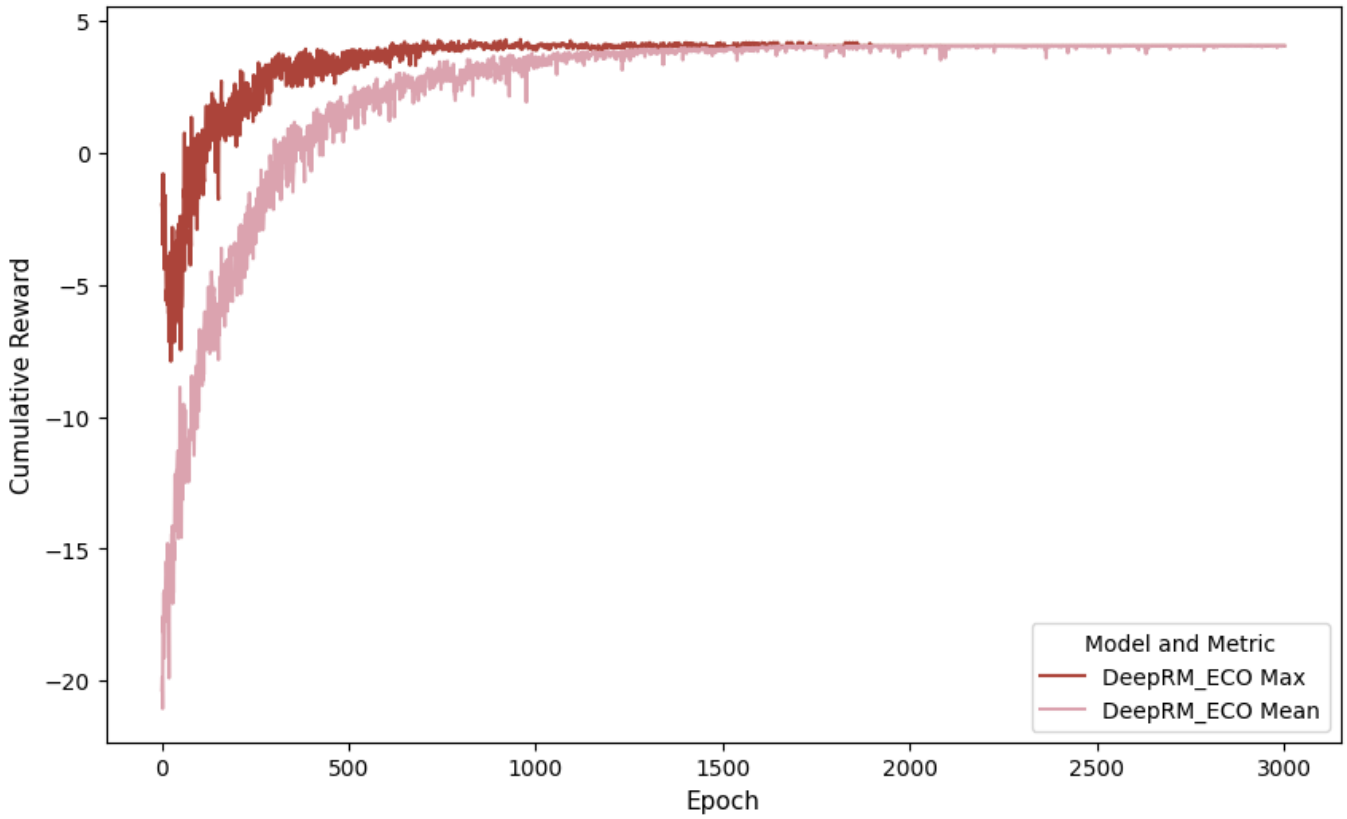

Figure 12: Mean and maximum cumulative reward of DeepRM_ECO over 3000 epochs with an anomaly rate of 0.1